ScholarPath — From ERD to EERD

Step 2: Enhancement

ScholarPath Team

Abstract

This is Part 2 of a four-part teaching series that walks the ScholarPath data model through the classical database-design pipeline: plain ERD → Enhanced Entity–Relationship diagram (EERD) → relational mapping → class diagrams. Building on the conceptual ERD established in Part 1, this document enriches that model into a full *Enhanced* ER diagram using the constructs of the Extended ER model (Elmasri & Navathe): specialization / generalization, disjointness and completeness constraints, weak entities, multivalued and derived attributes, and categories. We apply each construct to the actual ScholarPath schema — grounded in the live EF Core model snapshot and the domain entities — and we state one project-specific notation convention (solid versus dashed edges) so the diagrams cannot be misread by the defense committee. We close by previewing Part 3, the relational mapping.

Contents

1 Recap of Part 1 and the Goal of This Step **2**

2 The EER Toolbox (Elmasri & Navathe) **2**

2.1 Specialization and Generalization (ISA) 2

2.2 Disjointness and Completeness Constraints 3

2.3 Weak Entities 3

2.4 Multivalued and Derived Attributes 3

2.5 Categories (Union Types) 3

3 Applying the EER Model to ScholarPath: the USER Super-type **3**

3.1 The Two Constraints: Overlapping and Partial 4

3.2 How the Specialization Maps to the Physical Schema 4

4 Other EER Constructs in ScholarPath **5**

4.1 Weak Entities 5

4.2 Multivalued Attributes 5

4.3 Derived Attributes 5

5 A Project-Specific Notation: Solid vs Dashed Edges **6**

6 The Conceptual EERD and Its Subject-Area Clusters **6**

6.1 The Full Conceptual EERD (Chen / Elmasri Notation) 6

6.2 Identity, Access and Profile 7
6.3 Scholarships, Applications and Documents 9
6.4 Consultant Booking and Ratings 11
6.5 Community and Chat 13
6.6 Resources and Notifications 15
6.7 AI, Knowledge Base and Platform 17

7 Conclusion and Next Step **19**

1 Recap of Part 1 and the Goal of This Step

Part 1 (*ScholarPath — The Entity–Relationship Diagram*) established the plain conceptual ERD: it identified the core entity types of the platform — `USERS`, `SCHOLARSHIP`, `APPLICATION`, `BOOKING`, `FORUM_POST`, `RESOURCE`, `NOTIFICATION`, `AI_INTERACTION` and their neighbours — together with the relationship types that connect them and the cardinality ratios (1:1, 1:N, M:N) on each. That diagram answered the question *"what things exist and how are they related?"* at the level any introductory ER textbook would recognise: rectangles for entities, diamonds for relationships, ovals for attributes.

A plain ERD, however, cannot express several facts that are genuinely true of ScholarPath. It has no symbol for the idea that a single principal can simultaneously be a Student *and* a Consultant; it cannot say that an education entry has no independent existence without its owning profile; and it cannot distinguish an attribute that is physically stored from one that is *computed* on the fly. These are exactly the gaps the Enhanced ER model was designed to fill.

Goal of Step 2. Take the Part 1 ERD and *enrich* it into an EERD by adding the four "enhancement" constructs — specialization/generalization, weak entities, multivalued attributes, and derived attributes — plus one project-specific edge-style convention. Every enhancement in this document is grounded in the real model: the source of truth is the live EF Core model snapshot `server/src/ScholarPath.Infrastructure/Migrations/ApplicationDbContextModelSnapshot.cs` together with the domain entities under `server/src/ScholarPath.Domain/Entities/`. We do not invent entities or columns. The conceptual EERD is drawn in authentic Chen / Elmasri notation; the companion documents `02-RELATIONAL-MAPPING.md` (Part 3) and the class-diagram document (Part 4) continue from here.

2 The EER Toolbox (Elmasri & Navathe)

The Extended (or Enhanced) ER model adds a small, well-defined vocabulary on top of the basic ERD. We summarise the constructs we will use, then apply them in Section 3.

2.1 Specialization and Generalization (ISA)

A **superclass** (super-type) is an entity type whose members can be divided into **subclasses** (sub-types) that share the superclass's attributes but add their own. The relationship between a subclass and its superclass is the **IS-A** relationship, drawn in Elmasri notation with a circle joined to the superclass and a subset symbol ($\subset$) on each subclass line.

- **Specialization** is the top-down act of defining subclasses of a superclass (e.g. a `USER` is specialized into Student / ScholarshipProvider / Consultant / Admin).
- **Generalization** is the bottom-up act of recognising a common superclass for several entity types.

2.2 Disjointness and Completeness Constraints

Two orthogonal constraints qualify any specialization:

- **Disjointness** — written in the specialization circle:
  - **disjoint `{d}`**: an entity instance belongs to *at most one* subclass;
  - **overlapping `{o}`**: an entity instance may belong to *several* subclasses at once.
- **Completeness** — shown by the line from superclass to circle:
  - **total** (double line): *every* superclass instance must belong to at least one subclass;
  - **partial** (single line): a superclass instance *need not* belong to any subclass.

2.3 Weak Entities

A **weak entity type** cannot be identified by its own attributes alone; its existence and identification depend on an *owner* (identifying) entity, through an *identifying relationship*. A weak entity has only a **partial key** (discriminator) and is drawn with a double rectangle. In a relational schema, weak entities become child tables whose primary key includes (or whose existence is gated by) the owner's key, typically with a `CASCADE` delete rule.

2.4 Multivalued and Derived Attributes

- A **multivalued** attribute can hold a *set* of values for one entity instance (double oval). Classic relational mapping breaks it out into a separate table; a pragmatic modern alternative is to store the set as a JSON-encoded column.
- A **derived** attribute (dashed oval) is *computed* from other stored data rather than stored independently — e.g. an average, a count, or a percentage.

2.5 Categories (Union Types)

A **category** (or union type) is a subclass whose members are drawn from the union of *several distinct* superclasses — the EER answer to a "polymorphic" reference that can point at one of many entity types. ScholarPath has two such polymorphic references (the knowledge-base source and the audit-log target, Section 6.7); rather than model them as a formal category with a surrogate key, the platform realises them honestly as *loose* polymorphic id columns, which the next section makes explicit.

3 Applying the EER Model to ScholarPath: the USER Super-type

The single most important enhancement in ScholarPath is the specialization of the central `USER` entity. Conceptually a `USER` is a superclass specialized into four subclasses — **Student**, **ScholarshipProvider**, **Consultant**, and **Admin** — each adding its own role-specific attributes (Figure 1).

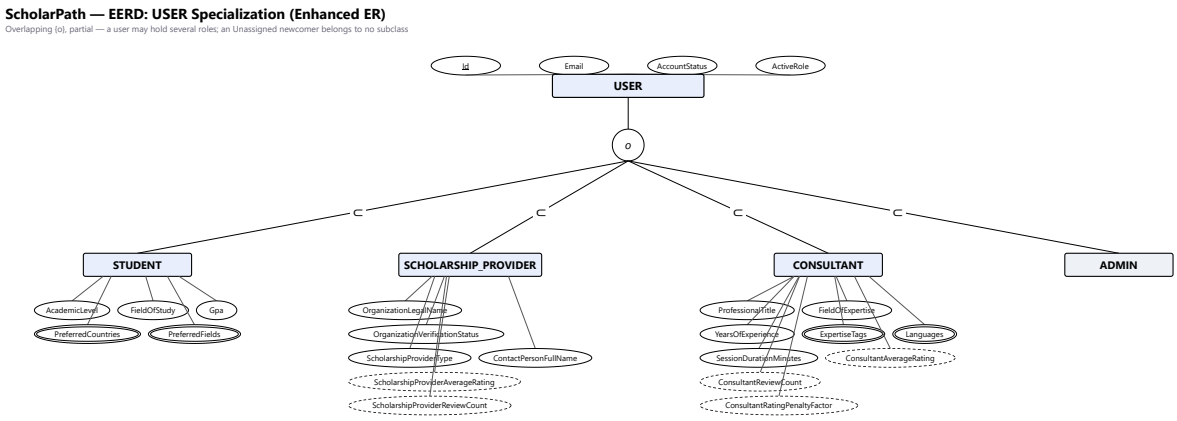


Figure 1: The `USER` super-type specialized into Student, ScholarshipProvider, Consultant and Admin in authentic Elmasri & Navathe EER notation. The specialization circle carries `o` (overlapping); the single line from `USER` to the circle marks partial participation; each subset symbol ($\subset$) reads "*subclass* IS-A USER".

3.1 The Two Constraints: Overlapping and Partial

Overlapping `{o}`. The specialization is *overlapping*, not disjoint: a user may hold *more than one* role at the same time. The canonical case is a Student who is later upgraded to Consultant and keeps both roles, so the same principal participates in the Student and Consultant subclasses simultaneously. This rules out the disjoint `{d}` constraint.

Partial. The specialization is *partial*, not total: a freshly-registered user starts in the `Unassigned` state and belongs to *no* subclass until role selection / onboarding completes. Because at least one valid `USER` instance (an unassigned newcomer) belongs to no subclass, the participation of the superclass in the specialization is partial (single line to the circle).

3.2 How the Specialization Maps to the Physical Schema

The EER picture above is the *conceptual* view. Its physical realization in the ScholarPath schema is deliberately a **single-table** mapping, not the textbook table-per-subclass:

- There is **no table-per-subclass**. The superclass is the single `Users` table.
- Role membership is the **`UserRoles`** join — the ASP.NET Identity many-to-many between `Users` and `Roles`. The seeded roles are exactly `Admin`, `Student`, `ScholarshipProvider`, `Consultant`, `Unassigned`. Because the join allows several rows per user, it is precisely what makes the specialization *overlapping*.
- The role-specific attributes — Student academic fields, ScholarshipProvider organization fields, Consultant expert fields — are *all columns on the single `UserProfiles` table*. For any given user, the columns belonging to the inapplicable sub-types are simply left `NULL`. This is the single-table (one relation for the whole hierarchy) realization of the specialization.

So the "Enhanced" modelling honestly captures the dual-role reality, while the storage strategy keeps a single principal row — consistent with the `UserProfiles` columns seen in Section 6.2 (`AcademicLevel`, `Gpa`, `OrganizationLegalName`, `SessionDurationMinutes`, `OrganizationVerificationStatu

4 Other EER Constructs in ScholarPath

Beyond specialization, three more enhancement constructs appear throughout the model.

4.1 Weak Entities

Several child entity types have no independent existence and are owned by a parent through an identifying relationship with a `CASCADE` delete rule. In the ScholarPath model the weak entities are exactly the cascade-owned children:

- **Education entries** (`EDUCATION_ENTRY`) depend on the owning `USER_PROFILE` — an education row is meaningless without its profile.
- **Scholarship child rows** (`SCHOLARSHIP_CHILD`, cascade) and **application child rows** (`APPLICATION_CHILD` — status history and notes, cascade) exist only relative to their parent `SCHOLARSHIP` / `APPLICATION`.
- **Upgrade-request attachments** (`UPGRADE_REQUEST_FILE`, `UPGRADE_REQUEST_LINK`) are attached to, and identified by, their parent `UPGRADE_REQUEST`.
- **Forum-post attachments** (`FORUM_POST_ATTACHMENT`) are owned by their `FORUM_POST`; **messages** (`MESSAGE`) are owned by, and ordered within, their `CONVERSATION`.
- **Resource chapters** (`RESOURCE_CHAPTER`) belong to a `RESOURCE`, and **per-chapter progress rows** (`RESOURCE_PROGRESS_CHILD`) belong to a `RESOURCE_PROGRESS`.

Two look-alikes that are *strong*, not weak. It is tempting to draw `SESSION_RECORDING` and `NOTIFICATION_PREFERENCE` as weak entities of `BOOKING` and `USER`, but both are **strong**: each carries its own independent `Id` key and its owner foreign key is `Restrict` (not `Cascade`), so neither is existence-dependent in the identifying sense. They are drawn as ordinary single-rectangle entities.

4.2 Multivalued Attributes

The Student subclass's **preferred countries** is a true multivalued attribute — a student may target several destination countries. Rather than break it into a separate table, ScholarPath stores the set as a JSON-encoded column on `UserProfiles` (mirroring `ExpertiseTagsJson` for consultants). This is the pragmatic JSON realization of a multivalued attribute, drawn in the conceptual diagram as the double-oval `PreferredCountries «multi»`.

4.3 Derived Attributes

Four attributes in the model are *computed*, not independently authored, and appear in the conceptual diagram as dashed ovals:

- `ScholarshipProviderAverageRating` — the provider's mean rating, derived from its `SCHOLARSHIP_PROVIDER`
- `UpvoteCount` — a forum post's net upvotes, cached on `FORUM_POST` but derived from its `VOTES` relationship.
- `ProfileCompletenessPercent` — a profile-fill percentage on `USER_PROFILE`, derived from which fields are populated.
- `CostUsd` — the per-call AI cost on `AI_INTERACTION`, derived from the prompt / completion token counts and the model's price.

5 A Project-Specific Notation: Solid vs Dashed Edges

ScholarPath adds one convention beyond the standard EER vocabulary, and it is important enough to state explicitly so the diagrams cannot be misread.

Line style	Meaning in this project
Solid (`--`)	A **database-enforced foreign key** — a real `FOREIGN KEY` constraint with a declared delete rule (`Restrict`, `Cascade` or `SetNull`).
Dashed (`..`)	An **application-enforced "loose" reference** — a `Guid` column that *semantically* points at another row but has **no FK constraint** in the schema. Referential integrity is guaranteed by application code, not by the database.

The platform deliberately uses loose references for high-volume, audit, log and analytics tables — notifications, notification preferences, votes, flags, bookmarks, chat participants, audit log, and AI interactions — so that deleting or anonymising a user never cascade-breaks history. Modelling them honestly is part of an accurate EERD; for example, `NOTIFICATION` and the *user* leg of `VOTES`, `FLAGS` and `BOOKMARKS` carry *zero* DB-enforced foreign keys back to `USER`, decoupling those high-volume rows from the principal for retention.

Note for examiners. In *standard* crow's-foot notation a solid line denotes an *identifying* relationship and a dashed line a *non-identifying* one. Here we deliberately **repurpose** that visual distinction to mean **DB-enforced FK (solid)** versus **application-enforced loose reference (dashed)**. This is a project-specific convention, stated up front so the diagrams cannot be misinterpreted.

6 The Conceptual EERD and Its Subject-Area Clusters

6.1 The Full Conceptual EERD (Chen / Elmasri Notation)

Figure 2 is the consolidated conceptual EERD in authentic Chen / Elmasri notation: rectangles are entity types, diamonds are relationship types, ovals are attributes, underlined ovals are keys, double ovals are multivalued attributes, dashed ovals are derived attributes, double lines mark total participation and single lines partial participation, and `1 / N` are the cardinality ratios. It is the master conceptual model from which the six per-cluster detail diagrams below are drawn.

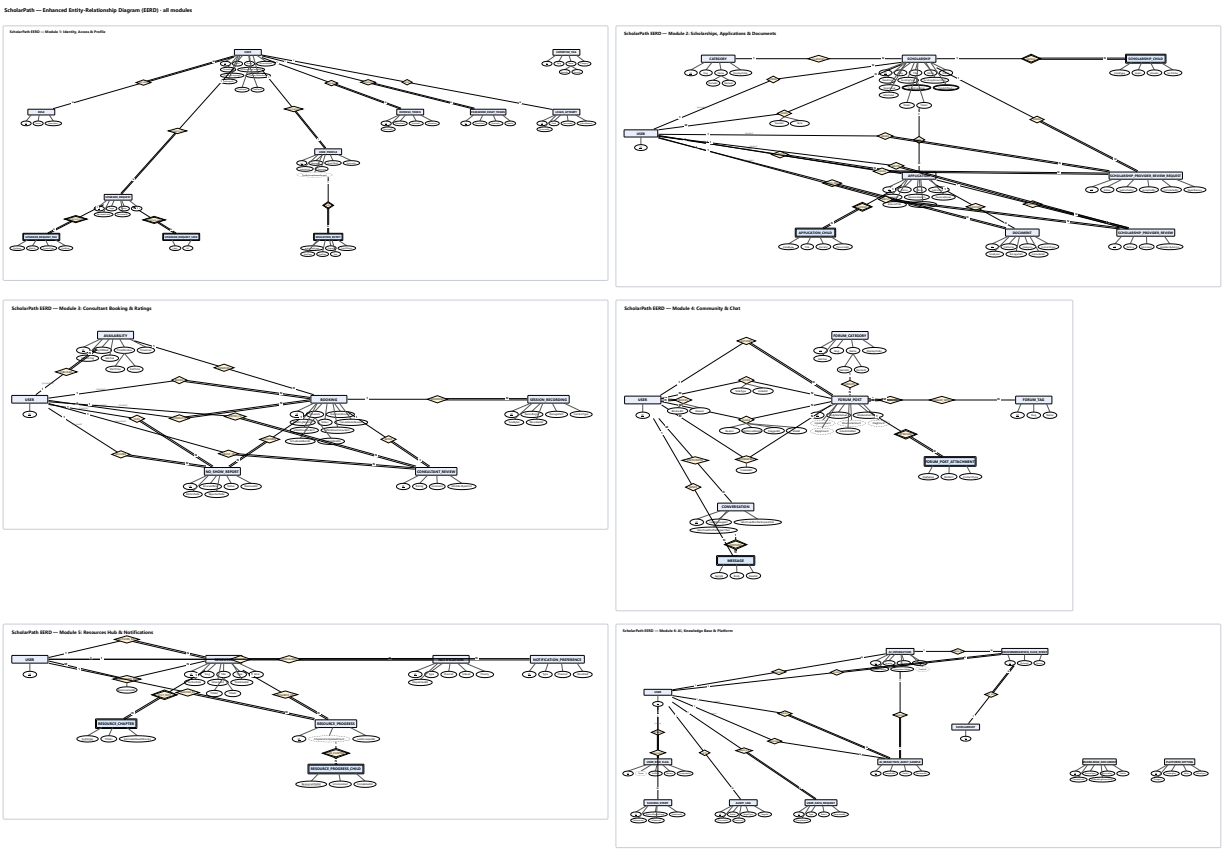


Figure 2: ScholarPath conceptual EERD (the "atlas" overview) in Chen / Elmasri notation. Everything radiates from `USER`; note the multivalued `PreferredCountries` and the derived `ScholarshipProviderAverageRating`, `UpvoteCount`, `ProfileCompletenessPercent` and `CostUsd`.

6.2 Identity, Access and Profile

This cluster holds the `USER` superclass, the `ROLE` membership (the `HAS_ROLE` M:N relationship) that realises the overlapping specialization, the 1:1 `USER_PROFILE` (carrying every role's attributes as nullable columns), the weak `EDUCATION_ENTRY`, the auth tokens (`REFRESH_TOKEN`, `PASSWORD_RESET_TOKEN`, `LOGIN_ATTEMPT`) and the `UPGRADE_REQUEST` flow with its weak file / link children. The standalone `EXPERTISE_TAG` taxonomy is referenced only through a denormalized JSON column, so it carries no relationship edge.

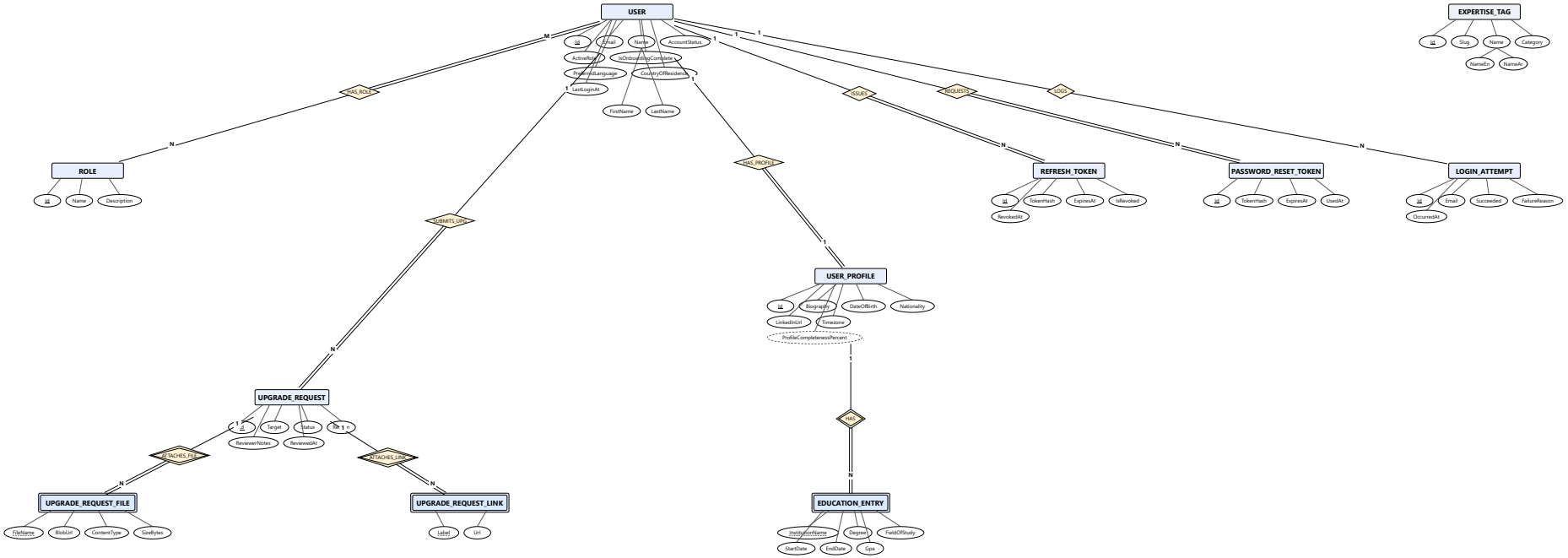


Figure 3: Module 1 — Identity, Access & Profile cluster (Chen / Elmasri notation).

6.3 Scholarships, Applications and Documents

The academic core: `CATEGORIES` classify `SCHOLARSHIPS` (owned optionally by a ScholarshipProvider), students `SUBMIT` `APPLICATIONS`, and `DOCUMENTS` support applications. Expert feedback is modelled by `SCHOLARSHIP_PROVIDER_REVIEW_REQUESTS` (raised by a student, fulfilled by a provider) and the resulting 1:1 `SCHOLARSHIP_PROVIDER_REVIEWS`. Under free mode there is no payment on this path. Note the weak `SCHOLARSHIP_CHILDREN` and `APPLICATION_CHILDREN` (cascade), the `SAVES` M:N "saved scholarships" relationship, and that the single-active-application rule is enforced in application code rather than by a DB index.

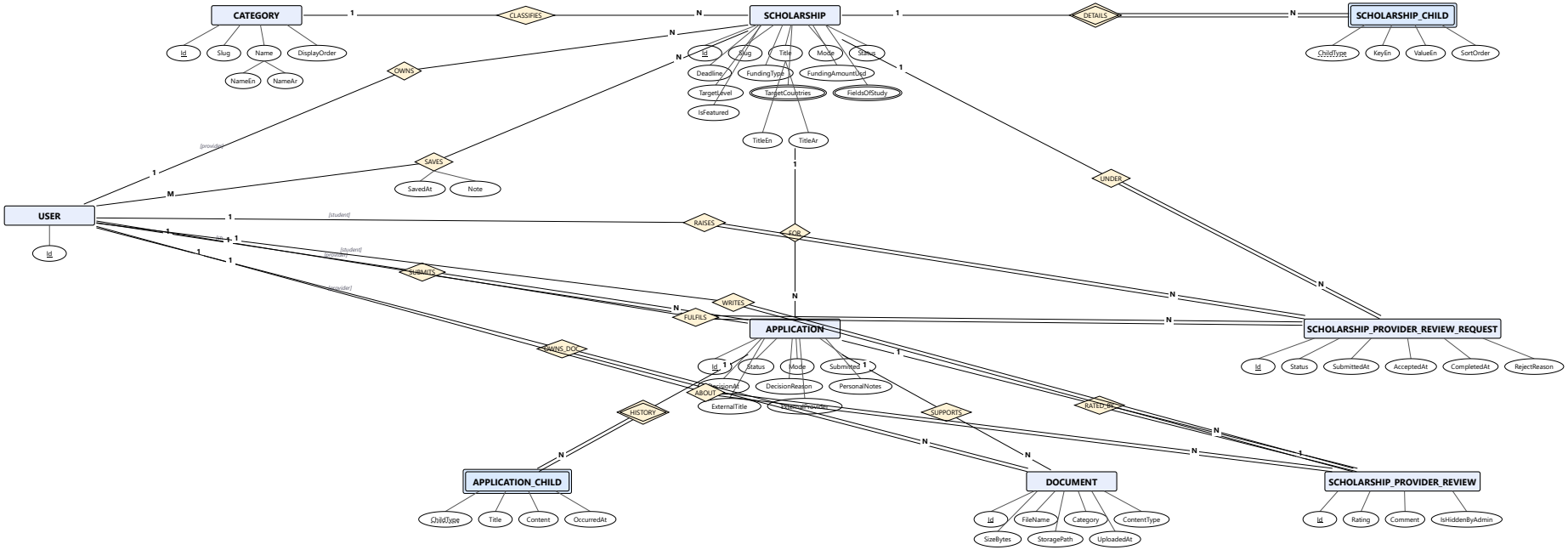


Figure 4: Module 2 — Scholarships, Applications & Documents cluster (Chen / Elmasri notation).

6.4 Consultant Booking and Ratings

Consultants PUBLISH AVAILABILITIES; students BOOK sessions that consultants CONDUCT; each BOOKING is rated by a 1:1 CONSULTANT_REVIEW and recorded as a SESSION_RECORDING. A NO_SHOW_REPORT can be filed by one party (reporter) accusing the other (accused) of failing to attend. Under free mode there is no settlement: the model carries *no* PAYMENT or PAYOUT entity. Note that SESSION_RECORDING is a *strong* entity (its BOOKING FK is Restrict, and it has its own Id), not a weak child.

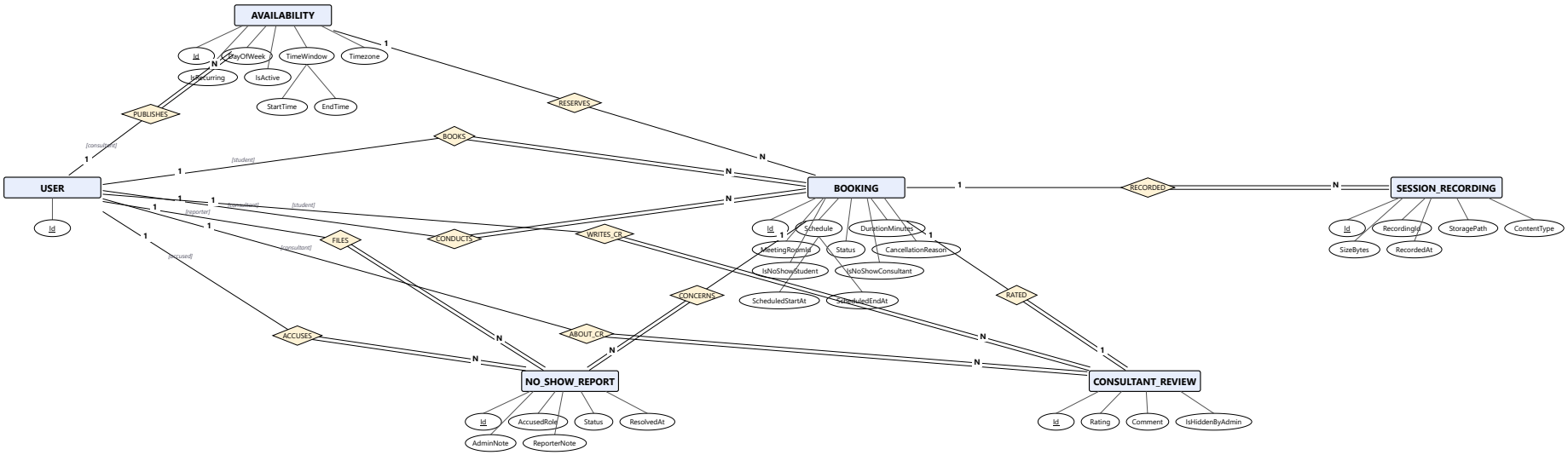


Figure 5: Module 3 — Consultant Booking & Ratings cluster (Chen / Elmasri notation).

6.5 Community and Chat

FORUM_POSTS (self-referencing for replies) are grouped by FORUM_CATEGORIES and tagged through the pure M:N TAGGED_WITH relationship (to FORUM_TAG). Votes, flags and bookmarks are modelled as M:N relationships *carrying their own attributes* (VOTES, FLAGS, BOOKMARKS) rather than as entity boxes — each with a composite "one per user per post" uniqueness. Weak FORUM_POST_ATTACHMENTS hang off a post. Private messaging is modelled by CONVERSATIONS (a unique participant pair) containing the weak MESSAGES; BLOCKS is a self-referencing M:N on USER. Votes, chat and blocks use loose user references.

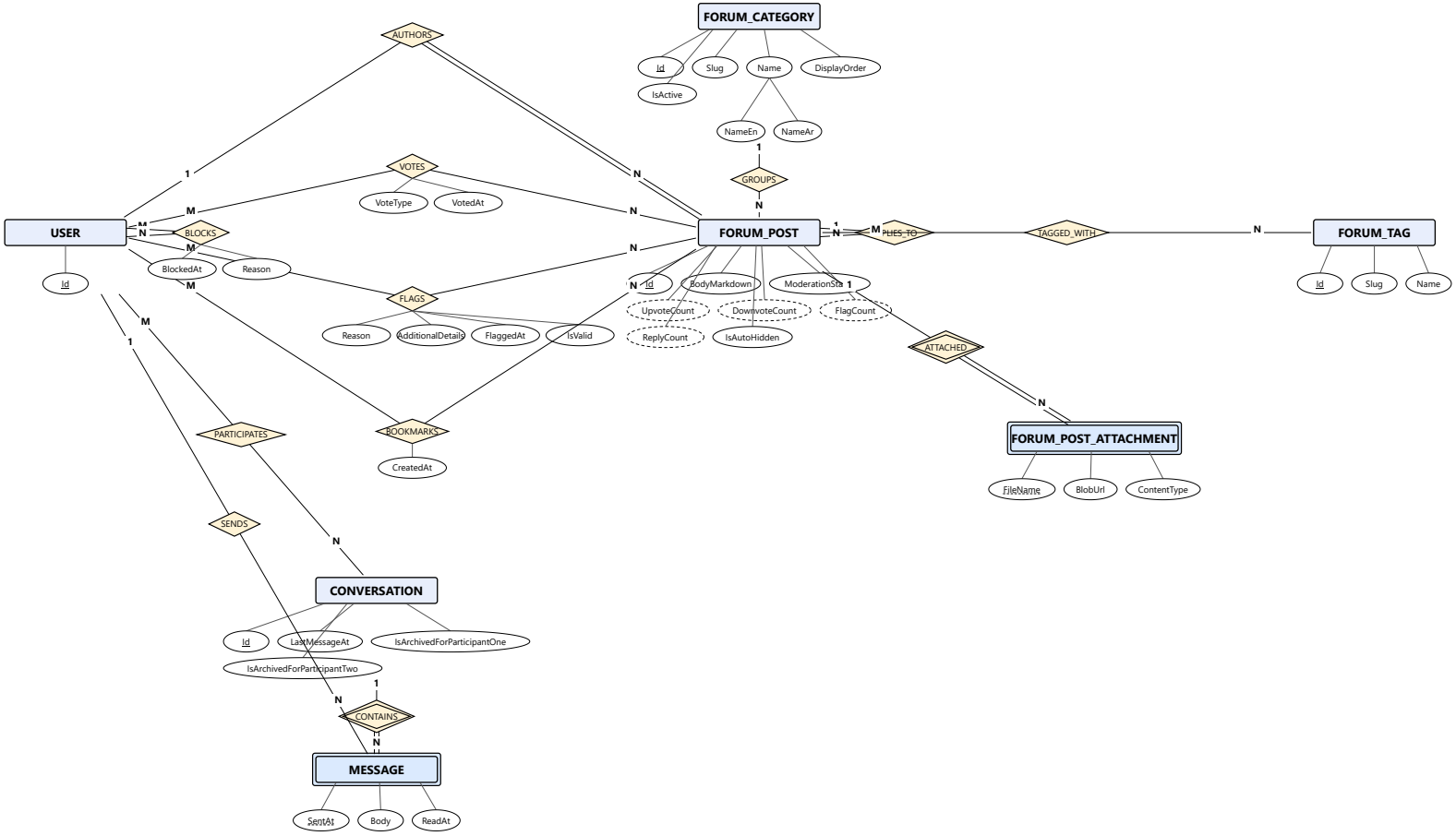


Figure 6: Module 4 — Community & Chat cluster (Chen / Elmasri notation).

6.6 Resources and Notifications

The Resources Hub: `RESOURCES` split into weak `RESOURCE_CHAPTERS`, with the per-user `BOOKMARKS_RES` M:N relationship and `RESOURCE_PROGRESS` (whose per-chapter progress rows, `RESOURCE_PROGRESS_CHILD`, are weak). Delivery is handled by `NOTIFICATIONS` (loose-referenced to the recipient, per Section 5) and per-type/-channel `NOTIFICATION_PREFERENCE`. Note that `NOTIFICATION_PREFERENCE` is a *strong* entity — it has its own `Id` and its `UserId` carries no FK/cascade — so it is drawn as a single rectangle, not a weak child.

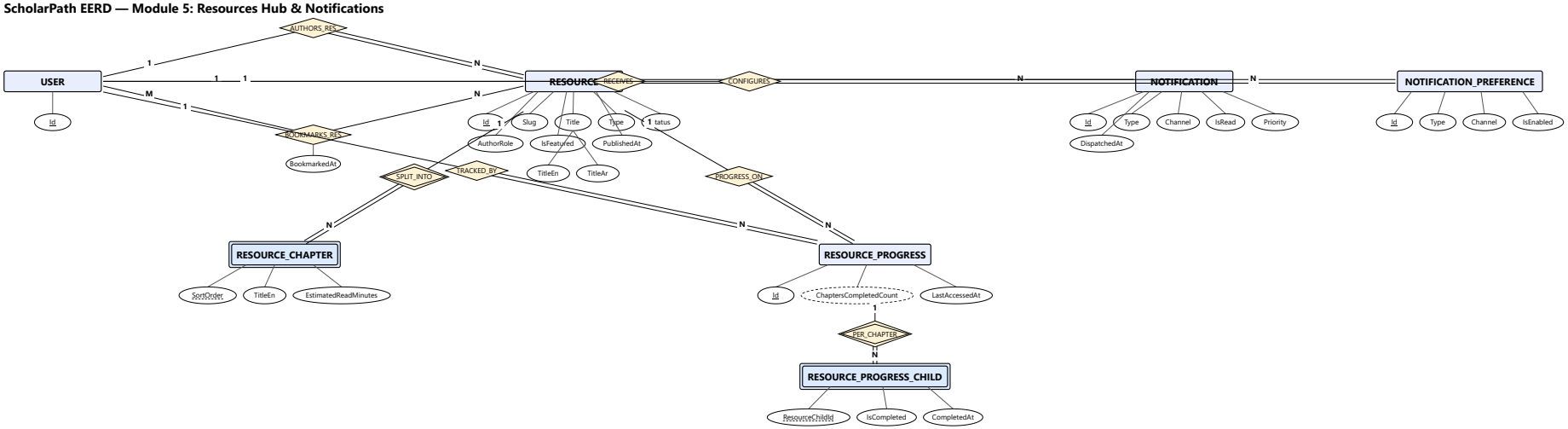


Figure 7: Module 5 — Resources Hub & Notifications cluster (Chen / Elmasri notation).

6.7 AI, Knowledge Base and Platform

The cross-cutting cluster: `AI_INTERACTIONS` (with the derived `CostUsd`) feed `RECOMMENDATION_CLICK_EVENTS` and are sampled by `AI_REDACTION_AUDIT_SAMPLES`; `USER_RISK_FLAGS` score users 1:1. `KNOWLEDGE_DOCUMENTS.S` and `AUDIT_LOGS.(TargetType, TargetId)` are *polymorphic* references — the category / union construct of Section 3 — realised as loose id columns because they can point at any entity type and so cannot be a single FK.

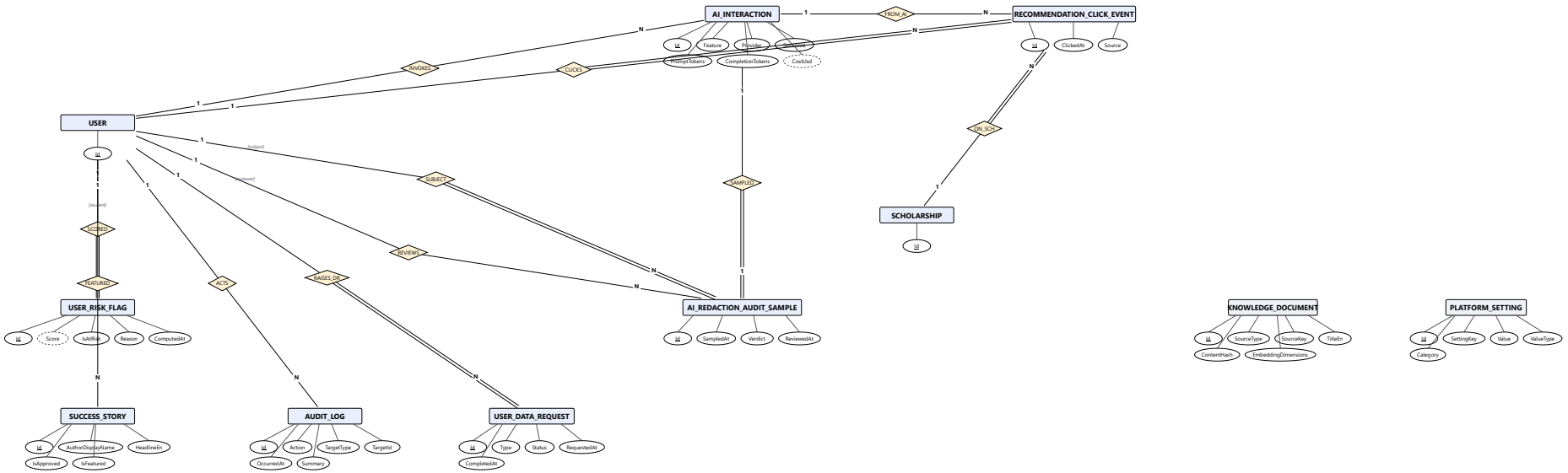


Figure 8: Module 6 — AI, Knowledge Base & Platform cluster (Chen / Elmasri notation).

7 Conclusion and Next Step

We have taken the plain ERD of Part 1 and enriched it into a full EERD. The headline enhancement is the `USER` specialization — *overlapping* (dual roles) and *partial* (an Unassigned newcomer belongs to no subclass) — physically realised as a single `Users` table plus a `UserRoles` join and nullable role columns on `UserProfiles`. We also added weak entities (education and child rows, upgrade attachments), multivalued attributes (preferred countries as JSON), derived attributes (`ScholarshipProviderAverageRating`, `UpvoteCount`, `ProfileCompletenessPercent`, `CostUsd`), categories (the two polymorphic references), and the project-specific solid-versus-dashed edge convention.

Next: Part 3 — Relational Mapping. Having a faithful conceptual EERD, we now translate it into actual relations: turning entities into tables, choosing primary and foreign keys, mapping the single-table specialization, breaking out (or JSON-encoding) multivalued attributes, materialising weak entities as cascade child tables, and turning each relationship's cardinality and participation into concrete key placement and delete rules. That table-by-table schema is the subject of `02-RELATIONAL-MAPPING.md`.